A PROJECT REPORT
ON
FOUNDATIONCOURSE (TINKERING & ELEMENTS DESIGN)

PLACEMENT ELIGIBILITY CHECKER

Submitted in partial fulfillment of the requirement for the award of Degree of Bachelor of Technology in Computer Science & Engineering

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted By:

Nandani kumari(24EARCS100)
Komal Agarwal(24EARCS077)
Muskan Singh(24EARCS099)
Kashish(24EARCS073)
Nency Jain(24EARCS102)

Session: 2025-26

Department of Computer Science & Engineering
ARYACOLLEGEOFENGINEERING& I.T.
SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

1. Introduction

1.1 Purpose

The purpose of the Placement Eligibility Checker is to help a college placement department manage student academic details, company requirements, and placement eligibility results through a desktop application.

1.2 Scope

The application stores student records and company-specific requirements. It compares a student's CGPA, percentage, backlog count, branch, attendance, and skills with the selected company's criteria, then displays an Eligible or Not Eligible result with the reasons.

1.3 Objectives

• Reduce manual effort when checking placement eligibility.

• Maintain consistent student and company records in a MySQL database.

• Apply company criteria consistently and show understandable rejection reasons.

• Provide search, filtering, update, delete, and result-summary features.

• Keep the code maintainable by separating the interface, business logic, and data-access layers.

1.4 Definitions

Term	Meaning
CGPA	Cumulative Grade Point Average recorded for a student.
Backlogs	Number of uncleared subjects, according to the college's definition.
CRUD	Create, Read, Update, and Delete operations.
JDBC	Java Database Connectivity, used to communicate with MySQL.
Eligibility result	Outcome of comparing student details with a company's criteria.

2. Overall Description

2.1 Product perspective

This is a Java desktop application. Java Swing provides the user interface, Java classes implement validation and eligibility rules, JDBC connects to MySQL, and MySQL stores persistent records.

2.2 User classes

Role	Main responsibilities
Administrator / Placement Officer	Manage students, companies, criteria, eligibility checks, and result summaries.
Student (optional)	View their own profile, browse company criteria, and check their own eligibility if student login is implemented.
Developer / Maintainer	Configure the environment, maintain code, troubleshoot issues, and manage database backups. This is not necessarily a separate app login.

2.3 Operating environment

• Java Development Kit (JDK 17 or another version selected by the project team).

• Java Swing desktop interface.

• MySQL Server 8.x or a compatible version.

• MySQL Connector/J JDBC driver.

• Windows, Linux, or macOS with a supported JDK and MySQL installation.

2.4 Assumptions and dependencies

• The college supplies accurate student records and approved company criteria.

• The database service is available when the application performs database operations.

• The placement officer decides the official meaning of CGPA, percentage, backlog, attendance, and branch requirements.

• Only authorized users should access administrative functions.

3. Functional Requirements

ID	Requirement	Description
FR-01	Authentication	The system shall verify the administrator's username and password before opening administrative features.
FR-02	Student registration	The system shall create student records with identity, contact, branch, CGPA, percentage, backlogs, attendance, and skills.
FR-03	Student search	The system shall search students by student ID, name, or branch.
FR-04	Student update/delete	The system shall allow authorized users to update or delete a selected student after validation and confirmation.
FR-05	Company management	The system shall add, view, update, search, and delete company records.
FR-06	Criteria management	The system shall store minimum CGPA/percentage, allowed backlogs, eligible branches, minimum attendance, and required skills for a company.
FR-07	Eligibility checking	The system shall compare a student's values against every configured criterion for the selected company.
FR-08	Reason display	The system shall show each criterion that passed or failed and produce a final eligibility status.
FR-09	Results and filtering	The system shall display eligibility results and filter them by company, student, branch, or status.
FR-10	Dashboard	The system should show summary counts such as total students, companies, eligible results, and not-eligible results.
FR-11	Database errors	The system shall display understandable messages when database access fails and shall not expose passwords or raw secrets.
FR-12	Data validation	The system shall reject missing required fields and out-of-range values before saving.

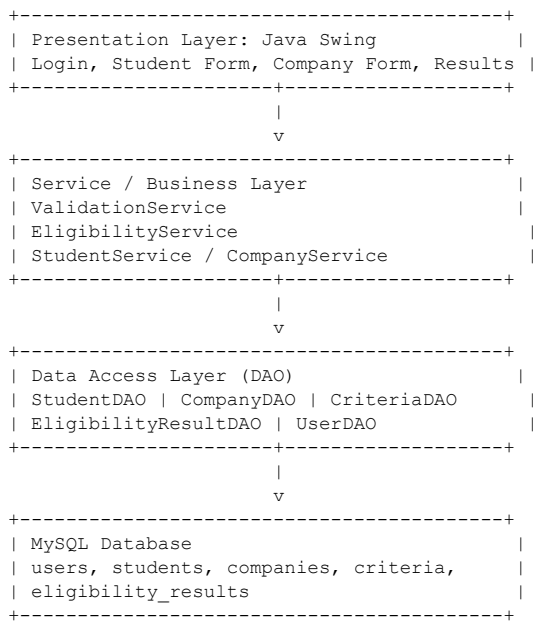
4. Non-Functional Requirements

Category	Requirement
Security	Use password hashing for stored passwords, parameterized PreparedStatement queries, and role checks for administrative actions.
Usability	Use clear labels, consistent buttons, readable tables, validation messages, and confirmation dialogs for destructive actions.
Reliability	Handle connection failures and exceptions without silently losing user data.
Performance	Common searches and eligibility checks should complete promptly for the expected college dataset.

Category	Requirement
Maintainability	Separate GUI, service/business logic, model classes, and DAO/database access.
Data integrity	Use primary keys, foreign keys where appropriate, unique constraints, and input validation.
Portability	Document JDK, MySQL, driver, and configuration requirements so the app can be set up on another machine.

5. System Architecture

Recommended layered architecture:



The Swing layer should not contain raw SQL or all eligibility rules. It should call service classes; service classes apply business rules and use DAO classes for database operations.

6. Core Modules

Module	Purpose	Key operations
Login & access	Restrict administrative features to authorized users.	Login, logout, credential validation.
Student management	Maintain academic and personal student data.	Register, view, search, update, delete.
Company management	Maintain recruiters and company information.	Add, list, search, update, delete.
Eligibility criteria	Store each company's placement rules.	Set thresholds, branches, backlog limits, attendance, skills.
Eligibility engine	Compare student data with company rules.	Evaluate every criterion, determine status, produce reasons.
Results & reports	Present outcomes clearly.	View, filter, summarize, export if implemented.
Database / configuration	Connect to MySQL safely.	Connection setup, exception handling, backup instructions.

7. Use Cases

Use case	Primary actor	Main flow
Log in	Placement Officer	Enter credentials → validate → open dashboard.
Register student	Placement Officer	Open form → enter details → validate → save→ show confirmation.
Manage company criteria	Placement Officer	Select company → enter criteria → validate → save criteria.
Check eligibility	Placement Officer / Student if supported	Select student and company → load data → compare criteria → show status and reasons.
Search records	Placement Officer	Enter keyword/filter → retrieve matching records → display table.
Update/delete record	Placement Officer	Select record → edit or confirm deletion → update database → refresh table.

8. UML Design

8.1Use-casediagram(text representation)

```
Placement Officer
 |----> (Log in)
 |----> (Manage Students)
 |----> (Manage Companies)
  |----> (Manage Eligibility Criteria)
|----> (Check Eligibility)
 |----> (View / Filter Results)  |----
> (View Dashboard)

Student (optional login)
 |----> (View Own Profile)  |---
-> (View Company Criteria)  |---
-> (Check Own Eligibility)
```

8.2 Class diagram (simplified)

```
User
- userId: int
- username: String
- passwordHash: String
- role: String

Student
- studentId: int
- name: String
- email: String
- branch: String
- cgpa: double
- percentage: double
- backlogs: int
- attendance: double
- skills: String

Company
- companyId: int
- name: String
- location: String
- jobRole: String

EligibilityCriteria
- criteriaId: int
- companyId: int
- minCgpa: double
- minPercentage: double
- maxBacklogs: int
- minAttendance: double
- eligibleBranches: String
- requiredSkills: String

EligibilityResult
- resultId: int
-  studentId:  int
-  companyId:  int
-  status:  String
- reasons: String
```

```
- checkedAt: datetime

EligibilityService
+ checkEligibility(student, company, criteria): EligibilityResult

StudentDAO / CompanyDAO / CriteriaDAO / ResultDAO
+ create(...)
+ findById(...)
+ update(...)
+ delete(...)
```

8.3 Sequence diagram: eligibility check

```
PlacementOfficer -> ResultsUI: select student and company
ResultsUI -> StudentService: getStudent(studentId)
StudentService -> StudentDAO: findById(studentId)
StudentDAO -> MySQL: SELECT student
MySQL --> StudentDAO: student record
ResultsUI -> CompanyService: getCompanyCriteria(companyId)
CompanyService -> CriteriaDAO: findByCompany(companyId)
CriteriaDAO -> MySQL: SELECT company criteria
MySQL --> CriteriaDAO: criteria record
ResultsUI -> EligibilityService: check(student, criteria)
EligibilityService --> ResultsUI: status + reasons
ResultsUI -> ResultDAO: save result (if enabled)
ResultDAO -> MySQL: INSERT eligibility result
ResultsUI --> PlacementOfficer: display status and reasons
```

9. Project Folder Structure

```
PlacementEligibilityChecker/
■■■ src/
■   ■■■ com/placementchecker/
■       ■■■ Main.java
■       ■■■ config/
■       ■   ■■■ DBConnection.java
■       ■■■ model/
■       ■   ■■■ User.java
■       ■   ■■■ Student.java
■       ■   ■■■ Company.java
■       ■   ■■■ EligibilityCriteria.java
■       ■   ■■■ EligibilityResult.java
■       ■■■ dao/
■       ■   ■■■ UserDAO.java
■       ■   ■■■ StudentDAO.java
■       ■   ■■■ CompanyDAO.java
■       ■   ■■■ CriteriaDAO.java
■       ■   ■■■ EligibilityResultDAO.java
■       ■■■ service/
■       ■   ■■■ AuthService.java
■       ■   ■■■ StudentService.java
■       ■   ■■■ CompanyService.java
■       ■   ■■■ EligibilityService.java
■       ■■■ ui/
■       ■   ■■■ LoginFrame.java
■       ■   ■■■ DashboardFrame.java
■       ■   ■■■ StudentForm.java
■       ■   ■■■ CompanyForm.java
■       ■   ■■■ EligibilityResultPanel.java
■       ■■■ util/
■           ■■■ ValidationUtil.java
■■■ database/
■   ■■■ schema.sql
■■■ docs/
■   ■■■ SRS.pdf
■   ■■■ diagrams/
■■■ README.md
■■■ .gitignore
```

If the project is built in a simple IDE without Maven, the same package structure can still be used. Keep database credentials outside committed source code and include a configuration example instead.

10. Database Design

Theproposed schemabelowisastarterdesign. Adjust columns to match the final forms and the eligibility rules approved by the placement department.

10.1 Tables

Table	Important columns	Purpose
users	user_id PK, username UNIQUE, password_hash, role, active	Login and role-based access.
students	student_id PK, roll_no UNIQUE, name, email, phone, branch, cgpa, percentage, backlogs, attendance, skills	Student profile and academic data.
companies	company_id PK, name, location, job_role, website, active	Recruiter information.
eligibility_criteria	criteria_id PK, company_id FK, min_cgpa, min_percentage, max_backlogs, min_attendance, eligible_branches, required_skills	Company-specific requirements.
eligibility_results	result_id PK, student_id FK, company_id FK, status, reasons, checked_at	History of eligibility checks.

10.2 Relationship rules

• One company may have one or more criteria versions if historical criteria are required; for a simple version, enforce one active criteria row per company.

• One student can have many eligibility results.

• One company can have many eligibility results.

• Use foreign keys to prevent orphaned results.

• Use DECIMAL columns for academic values where appropriate and validate allowed ranges in Java as well as in the database.

10.3 Example SQL schema

```
CREATEDATABASEplacement_checker;
USE placement_checker;

CREATE TABLE users (
 user_id INT AUTO_INCREMENT PRIMARY KEY,
username VARCHAR(60) NOT NULL UNIQUE,
password_hash VARCHAR(255) NOT NULL,
 role VARCHAR(30) NOT NULL DEFAULT 'ADMIN',
active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE students (
 student_id INT AUTO_INCREMENT PRIMARY KEY,
 roll_no VARCHAR(30) NOT NULL UNIQUE,
 name VARCHAR(120) NOT NULL,
 email VARCHAR(120),
 phone VARCHAR(20),
 branch VARCHAR(80) NOT NULL,
 cgpa DECIMAL(4,2) NOT NULL,
 percentage DECIMAL(5,2) NOT NULL,
 backlogs INT NOT NULL DEFAULT 0,
 attendance DECIMAL(5,2),
 skills TEXT,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CHECK (cgpa >= 0 AND cgpa <= 10),
 CHECK (percentage >= 0 AND percentage <= 100),
 CHECK (backlogs >= 0),
 CHECK (attendance IS NULL OR (attendance >= 0 AND attendance <= 100))
);

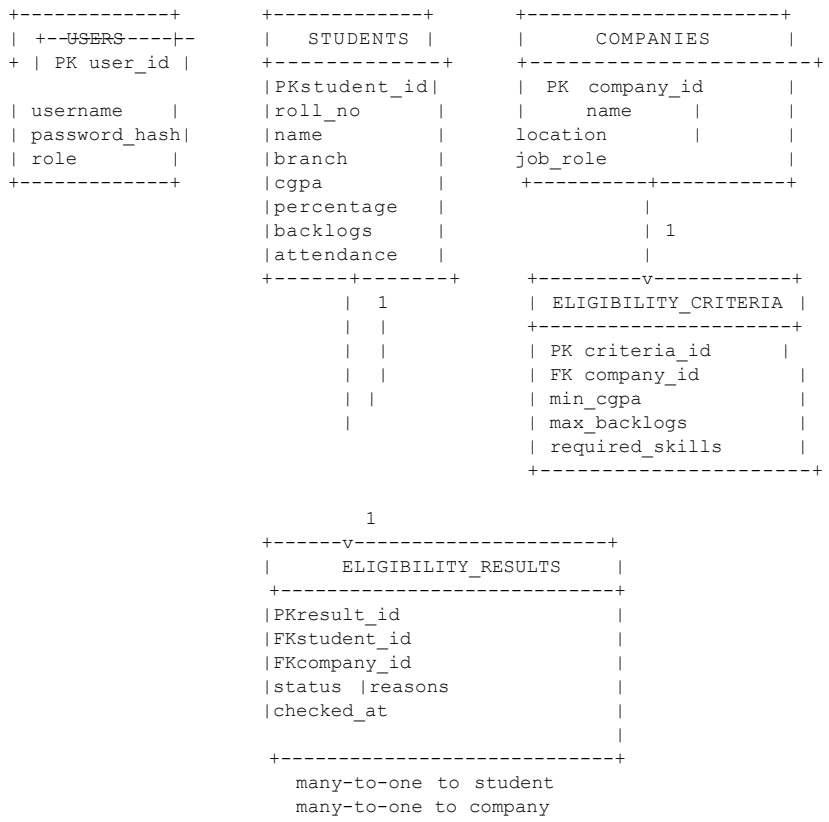
CREATE TABLE companies (
  company_id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(150) NOT NULL,
 location VARCHAR(150),
 job_role VARCHAR(120),
 website VARCHAR(255),
 active BOOLEAN NOT NULL DEFAULT TRUE
);
```

```
CREATE TABLE eligibility_criteria (
 criteria_id INT AUTO_INCREMENT PRIMARY KEY,
 company_id INT NOT NULL UNIQUE,
 min_cgpa DECIMAL(4,2),
 min_percentage DECIMAL(5,2),
 max_backlogs INT,
 min_attendance DECIMAL(5,2),
 eligible_branches TEXT,
 required_skills TEXT,
 FOREIGN KEY (company_id) REFERENCES companies(company_id)
);

CREATE TABLE eligibility_results (
 result_id INT AUTO_INCREMENT PRIMARY KEY,
 student_id INT NOT NULL,
 company_id INT NOT NULL,
 status VARCHAR(20) NOT NULL,
 reasons TEXT,
 checked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (student_id) REFERENCES students(student_id),
FOREIGN KEY (company_id) REFERENCES companies(company_id)
);
```

Note: CHECK constraint enforcement depends on the MySQL version. Keep Java-side validation even when database constraints are present. Store only password hashes, never plain-text passwords.

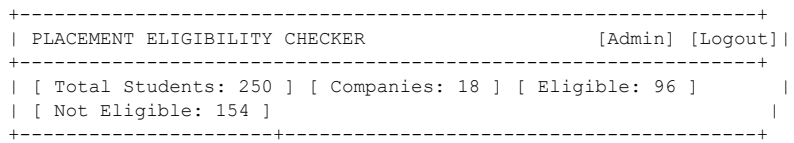
11. ER Diagram



Cardinalities: one student can have many results; one company can have many results; one company has one active criteria record in this simplified schema. Users are independent login records.

12. UI Mock-up

12.1Dashboard



```
| MENU                | QUICK ACTIONS                              |
| - Dashboard         | [ Register Student ] [ Add Company ]       |
| - Students          | [ Manage Criteria ] [ Check Eligibility]   |
| - Companies         |                                            |
| - Criteria          | Recent Eligibility Results                 |
| - Eligibility Check | Student | Company | Status | Date          |
| - Reports           | ...                                        |
 +--------------------+--------------------------------------+
```

12.2 Student registration form

```
+-----------------------------------------------------+
| STUDENT REGISTRATION                                |
 +---------------------------------------------------+
| Roll No:       [____________________]               |
| Full Name:     [____________________]               |
| Email:         [____________________]               |
| Phone:         [____________________]               |
| Branch:        [Select branch       v]              |
| CGPA:          [________]  Percentage:  [________]  |
| Backlogs:      [________]  Attendance: [________]   |
| Skills:        [_____________________________]      |
|                                                     |
| [ Save ] [ Update ] [ Clear ]    [ Search ]         |
 +---------------------------------------------------+
```

12.3 Eligibility check screen

```
+-----------------------------------------------------+
| CHECK PLACEMENT ELIGIBILITY                         |
 +---------------------------------------------------+
| Select Student: [Roll No / Name          v]         |
| Select Company: [Company Name            v]         |
|                                                     |
|                   [ CHECK ELIGIBILITY ]             |
+-----------------------------------------------------+
| RESULT: ELIGIBLE / NOT ELIGIBLE                     |
| CGPA requirement       PASS / FAIL                  |
| Percentage requirement PASS / FAIL                  |
| Backlog requirement    PASS  /  FAIL                |
| Branch requirement     PASS  /  FAIL                |
| Skills requirement     PASS / FAIL                  |
|                                                     |
| Explanation: [List of unmet requirements]           |
| [ Save Result ] [ Clear ] +-------------------------+
--------------------------+
```

These are wireframes, not final screenshots. Implement them with consistent fonts, aligned labels, validation feedback, accessible contrast, and confirmation dialogs for deletion.

13. Eligibility Logic

For each company criterion, compare the student's value with the configured threshold. A student is eligible only when all mandatory criteria pass. If a criterion is optional or missing, the placement department must define whether it should be ignored or treated as a failure.

```
failures = []

if student.cgpa < criteria.minCgpa:
    failures.add("CGPA below minimum")

if student.percentage < criteria.minPercentage:
    failures.add("Percentage below minimum")

if student.backlogs > criteria.maxBacklogs:
    failures.add("Too many backlogs")

if student.branch not in criteria.eligibleBranches:
    failures.add("Branch not eligible")

if criteria.minAttendance is set and
   student.attendance < criteria.minAttendance:
    failures.add("Attendance below minimum")

if required skills are not satisfied:
    failures.add("Required skills missing")

status = failures.isEmpty() ? "ELIGIBLE" : "NOT ELIGIBLE"
```

```
reasons = failures
```

The exact comparison policy must match college-approved rules. Define CGPA scale, treatment of active backlogs, missing attendance, skill matching, and whether criteria use AND or OR logic.

14. Validation and Error Handling

• Require student roll number, name, branch, and any fields the college marks mandatory.

• Validate CGPA against the chosen scale (for example, 0–10), percentage from 0–100, and non-negative backlog count.

• Check email/phone formats where required and prevent duplicate roll numbers.

• Use PreparedStatement for every SQL operation and close JDBC resources reliably.

• Show clear messages such as 'Student saved successfully' or 'Unable to connect to database'.

• Ask for confirmation before deleting records and refresh the table after successful changes.

15. Testing and Acceptance Criteria

Test ID	Scenario	Expected result
TC-01	Valid administrator credentials	Dashboard opens.
TC-02	Incorrect credentials	Login is denied with a safe message.
TC-03	Register valid student	Record is stored and appears in the student table.
TC-04	Enter CGPA above allowed scale	Validation blocks saving.
TC-05	Register duplicate roll number	Duplicate is rejected with a useful message.
TC-06	Student meets every mandatory criterion	Result is ELIGIBLE.
TC-07	Student fails one mandatory criterion	Result is NOT ELIGIBLE and the reason is shown.
TC-08	Update a student record	Correct record is updated and the table refreshes.
TC-09	Database is unavailable	Application handles the failure without crashing unexpectedly.
TC-10	Filter results by company/status	Only matching results are displayed.

16. Setup and Execution

• Install the JDK, MySQL Server, and an IDE such as Eclipse or IntelliJ IDEA.

• Create the database by executing database/schema.sql in MySQL Workbench or the MySQL command line.

• Add the MySQL Connector/J library to the project classpath or Maven dependencies.

• Configure the database URL, username, and password outside source control (for example, environment variables or a local properties file excluded by .gitignore).

• Run Main.java, create an authorized administrator account securely, and test the database connection.

• Commit source code, schema, setup instructions, and non-sensitive sample data to GitHub. Never commit real passwords or private student data.

17. Future Enhancements

• Separate student login and self-service dashboard.

• Export eligibility results to CSV or PDF.

• Email notifications and placement drive announcements.

• Resume upload and skill profile management.

• Audit logs, role-based permissions, and database backup automation.

• Migration to a web-based MERN or Spring Boot application if required in a later phase.

18. Conclusion

The Placement Eligibility Checker centralizes student and company data and provides a repeatable way to assess placement eligibility. The layered architecture, relational schema, validation, clear reasons, and test cases provide a foundation for a maintainable academic project. Final implementation should be reviewed against the placement department's actual rules before it is used for decisions.